

AI TRAINING – DAY 3 ASSIGNMENT

Python Operators & Strings: Manipulate → Extract → Clean → Format

Practical + MCQ + Debugging + String Utility Project

1. Learning Goals

- Understand arithmetic, comparison, logical, assignment and membership operators.
- Understand strings, indexing, slicing and common string methods.
- Use f-strings to create readable dynamic output.
- Identify common operator and string errors.
- Combine all topics to build a practical text utility.
- Explain your solution in your own words.

2. Assignment Rules

- Type the programs yourself. Do not copy complete solutions.
- Before running code, predict the output.
- Test strings with spaces, uppercase/lowercase, empty input and unexpected input.
- You may use AI to explain a concept or error after attempting it yourself.
- For every practical problem write Input → Processing → Output.
- Keep at least one error you faced and explain how you fixed it.

3. Part A – Operators Exploration

Create operators.py and experiment with each group. Write at least 2 examples for every group.

Operator group	Operators / your observation
Arithmetic	+ - * / // % ** Observation: Used for mathematical calculations.Ex:10+4=14, 2**3=8
Comparison	== != > < >= <= Observation: used to compare values and return True or False. :45==34->False, 10>=10->True
Logical	and or not Observation: Used to compare conditions .True and True-> True, not True->False
Assignment	= += -= *= /= Observation: Used for assign values to variables and update them. Ex: a=10,_ b=20 b/=10 ->2
Membership	in / not in Observation: Used to check a value is present in sequence or not .EX: L=[23,45,"apple"] "apple" in L->True,2 in L->False

Programs for operators:

```
a = 10
b = 6
c = 8
d = 4

##### Arithmetic Operators #####

# Addition
print(a + b)
print(c + d)

# Subtraction
print(a - b)
print(c - d)

# Multiplication
print(a * b)
print(c * d)

# Division
print(a / b)
print(c / d)

# Floor Division
print(a // b)
print(c // d)

# Modulo
print(a % b)
print(c % d)

# Exponentiation
print(a ** b)
print(b ** c)

##### Comparison Operators #####

# Equal to
print(a == b)
print(c == d)

# Not equal to
print(a != b)
print(c != d)

# Greater than
print(a > b)
```

```
print(c > d)

# Less than
print(a < b)
print(c < d)

# Greater than or equal to
print(a >= b)
print(c >= d)

# Less than or equal to
print(a <= b)
print(c <= d)

##### Assignment Operators #####

# Assignment
x = a
print(x)

# Add and assign
x = a
x += b
print(x)

# Subtract and assign
x = a
x -= b
print(x)

# Multiply and assign
x = a
x *= b
print(x)

# Divide and assign
x = a
x /= b
print(x)

# Floor divide and assign
x = a
x //= b
print(x)

# Modulo and assign
x = a
x %= b
```

```
print(x)

# Exponentiation and assign
x = a
x **= b
print(x)

##### Logical Operators #####

# AND
print(a > b and c > d)
print(a < b and c > d)

# OR
print(a > b or c < d)
print(a < b or c > d)

# NOT
print(not(a > b))
print(not(a < b))

##### Membership Operators #####

numbers = [10, 6, 8, 4]

# in
print(a in numbers)
print(b in numbers)

# not in
print(c not in numbers)
print(20 not in numbers)

##### Identity Operators #####

x = a
y = a
z = b

# is
print(x is y)
print(x is z)

# is not
print(x is not y)
print(x is not z)
```

```
##### Bitwise Operators #####
```

```
# Bitwise AND
```

```
print(a & b)
```

```
print(c & d)
```

```
# Bitwise OR
```

```
print(a | b)
```

```
print(c | d)
```

```
# Bitwise XOR
```

```
print(a ^ b)
```

```
print(c ^ d)
```

```
# Bitwise NOT
```

```
print(~a)
```

```
print(~b)
```

```
# Left Shift
```

```
print(a << b)
```

```
print(c << d)
```

```
# Right Shift
```

```
print(a >> b)
```

```
print(c >> d)
```

4. Part B – Operator Thinking

1. What is the difference between / and //?

Ans: / “gives the output float numbers but “ // “ gives the floor values.

2. When would % be useful in a real application?

Ans: When we need remainder.

3. What does ** do?

Ans: it used for exponentiation.

4. What is the difference between = and ==?

Ans: “=” is used for assignment operator and “==” comparison operator.

5. What is the difference between != and not?

Ans: “!=” is used for two values are different. “not” is used for reversing the Boolean value.

6. What is the result of 10 > 5 and 3 < 2? Explain.

Ans: False

7. What is the result of 10 > 5 or 3 < 2? Explain.

Ans: True

8.What does not True produce?

Ans:False

9.What does 'Python' in 'I am learning Python' return?

Ans:True

10.When would not in be useful?

Ans: “not in” is used to check whether the word in string or not.

5. Part C – String Basics Lab

Use your own name or another sentence and run these experiments.

11.Create a string and print it.

Ans:

```
word="deva"  
print(word)
```

12. Print its length using len().

Ans:

```
word="deva"  
print(len(word))
```

13. Print the first character using [0].

Ans:

```
word="deva"  
print((word[0]))
```

14. Print the last character using [-1].

Ans:

```
word="deva"  
print((word[-1]))
```

15. Print the first 3 characters using slicing.

Ans:

```
word="deva"  
print((word[0:3]))
```

16. Print the last 3 characters using slicing.

Ans:

```
word="deva"  
print((word[1:4]))
```

17. Print every second character.

Ans:

```
word="deva"
print((word[::2]))
```

18. Reverse the string using slicing.

Ans:

```
word="deva"
print((word[::-1]))
```

19. Try an invalid index. What error occurs?

Ans:

```
word="deva"
print((word[9])) #string index out of range
```

20 Try changing one character directly. What happens and why?

Ans:

```
word="deva"
word[0]='e'
print(word)
```

it cannot change because string is immutable.

6. Part D – String Methods Lab

Experiment with these methods and explain what each one does.

Method	Your example	What does it do?
lower()	"HARSHI". lower()->"harshi"	Converts upper case letters into lower case letters
upper()	"harshi".upper()->"HARSHI"	Converts lower case letters into upper case letters
title()	"harshi manthu" .title()->"Harshi Manthu"	Title() converts the first letter of each word into capital letter.
strip()	" Harshitha " .strip()->"Harshitha"	Remove spaces from beginning and ending of a string
replace()	Word=" I like ai". replace("java", "ai")	Replace one part of string with another
split()	"python,java,c,c#".split(",")->["python","java","c","c#"]	Split the string into list
join()	"/".join(["a","b","c"])->"a/b/c"	Join elements into string
startswith()	"Python".startswith("Py") -> True	Check whether the string start with particular value
endswith()	"Harshitha".endswith("tha")->True	Check whether the string ends with particular value
find()	"Python".find("t")->2	Find particular value index position in string
count()	"python program".count("p")	Count how many times a

7. Part E – Indexing & Slicing Challenge

Use: text = "Artificial Intelligence"

1. Print the first character.

Ans: Print(text[0])

2. Print the last character.

Ans: print(text[-1])

3. Print the first 10 characters.

Ans: print(text[:10])

4. Extract the word Intelligence.

A)

5. Reverse the complete string.

Ans: Print(text[::-1])

6. Print every second character.

Ans: print(text[::2])

7. Print characters from index 3 to 12.

Ans: print(text[3:13])

8. Find the index of the first 'i'.

Ans: print(text.index("i"))

9. Count the number of 'i' character.

Ans: print(text.count("i"))

10. Predict what text[100] will do, then run it.

Ans: print(text[100])

IndexError: string index out of range

8. Part F – f-Strings

1. Create name, age and city variables and display them in one sentence using an f-string.

```
name = "Devendra"
age = 21
city = "bayyavaram"
print(f"my name is {name} . i am {age} years old .i am from {city}")
```

#my is Devendra . i am 21 years old .i am from bayyavaram

2. Create a product, price and quantity and display a formatted bill.

```
product = "car"
price = 500000
quantity = 1
total = price * quantity
print(f"product : {product}")
print(f"quantity : {quantity}")
print(f"price : {price}")
print(f"total bill:{total}")
print("-----Thanks for visiting-----")
```

20. output: product : car

```
quantity : 1
price    : 500000
total bill : 500000
-----Thanks for visiting-----
```

3.Display student name, total marks and average.

```
student_name=input(str("enter student name :"))

subject1_marks=97
subject2_marks=78
subject3_marks=89
total=subject1_marks+subject2_marks+subject3_marks
average=total/3
print(f"Name      : {student_name}")
print(f"total marks : {total}")
print(f"average    : {average}")
```

21. output: Name : devendra
total marks : 264
average : 88.0

4.Use an f-string to display a calculation result.

```
a = 46

b = 2

result = a + b
print(f"The sum of {a} and {b} is {result}.")
```

output: The sum of 46 and 2 is 48.

5.Research decimal formatting and display a price with exactly 2 decimal places.

```
price = 89.536836
print(f"Price: {price:.2f}")
output: The price is 89.53
```

9. Part G – MCQs: Basic

1. Which operator means multiplication?

- A. x
- **B. ***
- C. %
- D. //

2. What does % return?

- A. Division
- B. Quotient
- **C. Remainder**
- D. Power

3. What is the first string index?

- **A.** 0
- B. 1
- C. -1
- D. first

4. What is len('Python')?

- A. 5
- **B.** 6
- C. 7
- D. Error

5. Which gets the last character?

- A. text[0]
- **B.** text[-1]
- C. text[last]
- D. text[1]

6. Which converts text to lowercase?

- **A.** lower()
- B. small()
- C. down()
- D. lowercase

7. Which removes leading/trailing spaces?

- A. trim()
- B. clean()
- **C.** strip()
- D. remove()

8. Which checks membership?

- A. inside
- **B.** in
- C. has
- D. contains

9. Which is an f-string?

- **A.** f"Hello {name}"
- B. "f Hello"
- C. format f(name)
- D. string(name)

10. Which compares equality?

- A. =
- **B.** ==

- C. ===
- D. equals

10. Part H – MCQs: Medium

1. What is `17 // 5`?

- A. 3
- B. 3.4
- C. 2
- D. 4

2. What is `17 % 5`?

- A. 3
- B. 2
- C. 3.4
- D. 0

3. What does `'Python'[1:4]` return?

- A. Pyt
- B. yth
- C. ytho
- D. tho

4. What does `'Python'[-3:]` return?

- A. tho
- B. hon
- C. yth
- D. Python

5. What does `' hello '.strip()` return?

- A. ' hello'
- B. 'hello '
- C. 'hello'
- D. Error

6. What does `'hello'.upper()` return?

- A. hello
- B. HELLO
- C. Hello
- D. Error

7. What does `'a,b,c'.split(',')` return?

- A. abc
- B. ['a','b','c']
- C. ('a','b','c')

- D. Error

8. Which checks if AI occurs in text?

- A. text.contains('AI')
- **B. 'AI' in text**
- C. text.has('AI')
- D. contains(text,'AI')

9. What happens with text[50] when text has 10 characters?

- A. None
- B. "
- **C. IndexError**
- D. False

10. Why use f-strings?

- **A. Insert values/expressions into strings cleanly**
- B. Make strings immutable
- C. Convert values to int
- D. Remove spaces

11. Part I – MCQs: Thinking

1. Which is true about strings?

- A. They can always be changed character-by-character
- **B. They are immutable**
- C. They only contain letters
- D. They cannot be sliced

2. Best way to clean ' PHANI gorantla ' for display?

- A. strip().title()
- B. title().strip()
- **C. Both A and B**
- D. clean()

3. What does text[::-1] do?

- A. Deletes text
- **B. Reverses text**
- C. Sorts text
- D. Gets first character

4. Best simple test for '@' in an email?

- **A. '@' in email**
- B. email == '@'
- C. email.contains('@')
- D. email.has('@')

5. `replace()` generally:

- **A.** Returns a transformed string
- B. Always changes the original string in place
- C. Converts to list
- D. Deletes the variable

6. `x='5', y=2`. Which gives numeric 10?

- A. `x*2`
- **B.** `int(x)*2`
- C. `str(x)*2`
- D. `x+2`

7. Which formats 12.5 as 12.50?

- **A.** `f'{price:.2f}'`
- B. `f'{price:2}'`
- C. `f'{price.float}'`
- D. `f'{price.decimal(2)}'`

8. What should usually happen before validating text entered with spaces?

- **A.** `strip()`
- B. `reverse()`
- C. `split()` always
- D. replace with None

9. A common cause of `IndexError` is:

- **A.** Invalid index
- B. Using an f-string
- C. `lower()`
- D. +

10. Why keep raw input and cleaned input conceptually separate?

- **A.** It makes transformations easier to test
- B. Python requires it
- C. Strings cannot be stored
- D. It prevents every error

12. Part J – 5 Practice Problems

Problem	Task	Topics
Practice 1 – Username Cleaner	Ask for a username, remove surrounding spaces, convert to lowercase and display the cleaned value.	<code>strip</code> + <code>lower</code> + f-string
Practice 2 – Name Formatter	Ask first and last names in random case and display a properly formatted full name.	<code>strip</code> + <code>title</code> + f-string
Practice 3 – Word Analyzer	Ask for a sentence. Display	<code>len</code> + indexing + count

	length, first character, last character, spaces count and vowel count. Do not use loops.	
Practice 4 – Email Inspector	Ask for an email. Check '@', check whether it ends with '.com' or '.in', and show the position of '@'.	in + endswith + find
Practice 5 – Product Code Extractor	For code IND-MOB-2026-001, extract country, category, year and serial using slicing.	indexing + slicing + f-string

For each: Input → Expected Output → Code → 2 test cases → 1 edge case → explanation.

Practice 1 – Username Cleaner:

Code:

```
username = input("Enter your username: ")
clean_username = username.strip()
lower_username = clean_username.lower()
print(f"The cleaned username is {lower_username}")
```

output:

Enter your username: Devendra

The cleaned username is devendra.

Practice 2 – Name Formatter:

CODE:

```
first_name_input = input("Enter your first name: ")
last_name_input = input("Enter your last name: ")
first_name = first_name_input.lower()
last_name = last_name_input.lower()
print(f"The full name is {first_name} {last_name}")
```

OUTPUT:

Enter your first name: Devendra

Enter your last name: Malepati

The full name is devendra malepati

Practice 2 – Word Analyzer:

Code:

```
sentence = input("Enter any sentence: ")
sentence_length = len(sentence)
first_character = sentence[0]
last_character = sentence[-1]
space_count = sentence.count(" ")
vowel_count = (sentence.lower().count("a") + sentence.lower().count("e") +
sentence.lower().count("i") + sentence.lower().count("o") + sentence.lower().count("u"))
print(f"Length of sentence is {sentence_length}")
```

```
print(f"The first character is {first_character}")
print(f"The last character is {last_character}")
print(f"{space_count} spaces are present in the sentence")
print(f"{vowel_count} vowels are present in the sentence")
```

output:

Enter any sentence: i like devara movie

Length of sentence is 19

The first character is i

The last character is e

3 spaces are present in the sentence

9 vowels are present in the sentence

Practice 4 – Email Inspector:

CODE:

```
email_id = input("Enter your email: ")
ends_with_com = email_id.endswith(".com")
print("email contains @?", "@" in email_id)
print("email ends with .com?", ends_with_com)
print("where @ symbol is located", email_id.find("@"))
```

output:

Enter your email: dinesh@email.com

email contains @? True

email ends with .com? True

where @ symbol is located 6

Practice 5 – Product Code Extractor

CODE:

```
product_code = input("Enter product code (IND-ELE-2025-045): ")
country = product_code[:3]
category = product_code[4:7]
year = product_code[8:12]
```

```
serial_number = product_code[13:]  
  
print(f"Country: {country}")  
print(f"Category: {category}")  
print(f"Year: {year}")  
print(f"Serial number: {serial_number}")
```

output:

Enter product code (IND-ELE-2025-045): AUS-ALE-2026-644

Country: AUS

Category: ALE

Year: 2026

Serial number: 644

13. Part K – Debugging Lab

Bug 1

```
name = input('Name: ')  
print(name[20])
```

Why can this fail? Fix it.

Corrected code / explanation:

Explanation: the code is correct .In that code it is trying to print 21 character if that name contain more than 21 character it will print. Other wise it wil give error index out of range.

Bug 2

```
age = '20'  
print(f'Age: {age + 1}')
```

Fix it.

Corrected code / explanation:

```
age = '24'  
print(f'Age: {int(age) + 1}')
```

Explanation: age contains the value is string .we cannot add integer to string.then we need convert the string to int and add value to it.

Bug 3

```
name = 'phani'  
age = 30  
print('Hello ' + name + age)
```

Identify the type problem and fix it.

Corrected code / explanation:

```
name = 'phani'
age = 30
print(f" hello {name} .you are {age} years old." )
```

Explanation: it gives error because age is integer value it cannot added to string .instead we can use f string to combine all values.

Bug 4

```
email = input('Email: ')
print(email.endswith('.com'))
```

Test with 3 inputs and explain.

Corrected code / explanation: the given code is correct .but it will gives the true output even the email is .com. the code doesn't care about any rules of email.

Ex-1:

Input:devendra@gmail.com

Output:True

Ex-2:

Input :devendra@gmail.

Output: False

Ex-3:

Input: .com

Output: True

Bug 5

```
code = 'IND-MOB-2026-001'
print(code[20])
```

Find the valid index range and fix the requirement.

Corrected code / explanation:

```
code = 'IND-MOB-2026-001'
print(code[15])
```

Explanation: length of the code is 16 characters .as index starts from 0 we can use upto 15 to print last character.

Bug 6

```
text = ' hello world '
print(text.upper().strip())
```

Does it work? Explain the order of operations.

Corrected code / explanation:

```
text = ' hello world '  
print(text.upper().strip()) #HELLO WORLD
```

Explanation: code will work .it will convert the string to upper case and then remove spaces at before and after of the string.

14. Part L – Main Project: Text-Cleaning & Profile Formatter

Build a practical Python program that accepts messy profile information and converts it into a clean, readable profile.

Scenario

A company receives profile data in inconsistent formats. Names may contain extra spaces, emails may use mixed cases, and cities may be entered randomly. Your program should clean the data and generate a professional profile summary.

Required Inputs

- Full name
- Email address
- Phone number
- City
- Job role
- Short bio

Must-Have Requirements

- Remove leading/trailing spaces.
- Format the full name using title().
- Convert email to lowercase.
- Format city and job role neatly.
- Calculate bio character count.
- Check whether the email contains '@'.
- Check whether the email ends with a supported domain such as .com or .in.
- Extract first name from the full name.
- Extract last name using string logic.
- Display the profile using f-strings.
- Show a small data-quality section.
- Test with messy uppercase/lowercase and extra spaces.

Expected Output Example

```
===== PROFILE =====  
Name           : Phani Gorantla  
First Name     : Phani  
Email          : phani@example.com  
Phone          : 9876543210  
City           : Hyderabad  
Role           : Software Developer
```

Bio Length : 42 characters

```
----- DATA QUALITY -----  
Email Valid : Yes  
=====
```

Do not hard-code the example. The program must work with any user's input.

CODE:

```
full_name=input("enter your name:")  
first_name=full_name.split()[0]  
email_address=input("enter your email:")  
phone_number=input("enter your phone number:")  
city=input("enter yor city name:")  
job_role=input("enter your job role:")  
short_bio=input("short bio:")  
print("=====Profile=====")  
print(f"Name      :{full_name.title().strip()}")  
print(f"First Name  :{first_name}")  
print(f"Email       :{email_address.lower()}")  
print(f"city        :{city.title()}")  
print(f"Phone number :{phone_number}")  
print(f"job role     :{job_role.strip()}")  
print(f"short bio    : {short_bio.strip()}")  
print("-----Data Quality-----")  
contains= '@' in email_address  
ends=email_address.endswith(".com")  
ends_with=email_address.endswith(".in")  
Email_valid=contains and ends or ends_with  
print(f"email valid : {Email_valid}")  
print("=====")
```

OUTPUT:

```
=====Profile=====
```

Name :Devendra Malepati

First Name :devendra

Email :deva@gmail.com

city :Kurichedu

Phone number :9391766657

job role :ethical hacker

short bio : fast learner

-----Data Quality-----

email valid : True

=====

15. Project Bonus Challenges

- Clean repeated spaces inside a name.
- Validate phone number length.
- Check that the full name contains at least two words.
- Generate initials from the name.
- Generate a username from the person's name.
- Create a one-line professional bio using an f-string.
- Display a simple data-quality percentage.

CODE:

```
full_name = input("Enter your full name: ")
email = input("Enter your email: ")
phone = input("Enter your phone number: ")
city = input("Enter your city: ")
role = input("Enter your job role: ")
bio = input("Enter your short bio: ")
cleaned_name = " ".join(full_name.split())
validated_phone = len(phone) == 10
full_name_check = len(full_name.split()) >= 2
words = cleaned_name.split()
word_count = len(cleaned_name.split())
initials = words[0][0].upper() + words[1][0].upper()
username = words[0].lower() + "@" + words[1].lower()
professional_bio = f"My name is {cleaned_name}. I am from {city}. I am interested in {role}."
total_fields = 6
filled_fields = filled = bool(full_name) + bool(email) + bool(phone) + bool(city) + bool(role) + bool(bio)
quality = (filled_fields / total_fields) * 100
print("=====results=====")
print("Clean Name:", cleaned_name)
print("Phone Length:", len(phone))
print("Phone Valid:", validated_phone)
print("Word Count:", word_count)
```

```

print("Name Valid:", full_name_check)
print("Initials:", initials)
print("Username:", username)
print("Professional Bio:", professional_bio)
print("Data Quality:", quality, "%")

```

OUTPUT:

Enter your full name: deva malepati
 Enter your email: deva@gmail.com
 Enter your phone number: 9391766867
 Enter your city: darsi
 Enter your job role: ethical hacker
 Enter your short bio: i am intrested in job
 =====results=====
 Clean Name: deva malepati
 Phone Length: 10
 Phone Valid: True
 Word Count: 2
 Name Valid: True
 Initials: DM
 Username: deva@malepati
 Professional Bio: My name is deva malepati. I am from darsi. I aminterested in ethical hacker.
 Data Quality: 100.0 %

16. Project Test Cases

Test	Input condition	Expected behaviour	Actual result
1	Normal clean input	Clean profile	Profile displayed corrected
2	Extra spaces around fields	Spaces removed	Extra spaces removed
3	Random uppercase/lowercase	Proper formatting	Name and username formatted properly
4	Invalid-looking email	Flag data quality	Data quality percentage shows the available data
5	Short/missing optional data	Program should handle it sensibly	Program accepts the input and displays the result

17. Explain Your Project

22. What is the difference between cleaning data and formatting data?

Ans : data cleaning is nothing but removing unwanted spaces and invalid values . data formatting means converting the string into required form.

23. Which string methods did you use and why?

Ans: I used split(), join(), upper(), and lower(), strip()

- split() : it separates words from string.

- `join()` : it joins words together.
- `upper()` :converts the letters to uppercase.
- `lower()` :converts letters to lowercase.
- `.strip()` : it removes the extra spaces at before and after of the string.

24. Where did you use indexing?

Ans :Used indexing at slicing and to get any characters from the string.

25. Where did you use slicing?

Ans: Slicing is used to extract required part from the string.

26. Why are f-strings useful here?

Ans:F-strings are useful because we can print string and integers together we can perform operation inside the fstring.

27. What error did you face?

Ans:at starting I had faced errors at which string methods want to use at finally I learn.

28. How did you test edge cases?

Ans:tested the programs input with many variations like which may cause errors.

29. If this processed 1 lakh profiles, what would you improve first?

Ans: data validation and memory usage.

18. Final Self-Check

Skill	Yes	Need Practice	Notes
Use arithmetic/comparison/logical operators	☒	☐	
Explain = vs ==	☒	☐	
Manipulate strings	☒	☐	
Use positive and negative indexing	☒	☐	
Use slicing	☒	☐	
Use string methods	☒	☐	
Use in/not in	☒	☐	
Use f-strings	☒	☐	
Debug IndexError/type problems	☐	☒	
Build the final text utility	☒	☐	

19. Submission Checklist

- Operator experiments
- Operator questions
- String exploration
- String-method table
- Indexing/slicing challenge
- f-string exercises
- 30 MCQ answers
- 5 practice programs
- Debugging Lab

- Text-Cleaning & Profile Formatter
- Project test cases
- Project explanation and self-check